University of Bologna
Master's Degree in Computer Science and Engineering

Lab-activity-05

Laboratory Report

Student:
Sajmir Buzi

Academic Year 2025/2026

1 Objective

The objective of this activity is to train a robot controller in ARGoS using a simple neural network and a genetic algorithm. The robot has to move as close as possible to a light source. Instead of writing all the movement rules manually, the controller uses the light sensor values as input and a set of evolved weights to decide the speed of the two wheels.
The implementation is divided into two main parts. The Lua files implement the neural network and the ARGoS controller, while the Python file implements the genetic algorithm used to search for good network weights.

2 Controller Description

The neural network is implemented in `nn.lua`. It has 24 input neurons, one for each light sensor of the robot, and 2 output neurons, one for the left wheel and one for the right wheel. The genome contains 50 values in total: 48 connection weights from the inputs to the outputs, plus 2 bias values.
At the beginning of the simulation, `controller-nn.lua` reads the genome from the environment variable `GENOME`. The values are stored as a comma-separated string, then they are converted into numbers and used to create the neural network.
During each simulation step, the controller reads the 24 light sensors and passes their values to the network. The network computes two output values using a sigmoid function, so the results are always between 0 and 1. These values are multiplied by the maximum velocity, set to `15`, and then assigned to the wheels.
In this way, the movement of the robot depends directly on the weights of the neural network. If the two outputs are similar, the robot moves almost straight. If one output is higher than the other, the robot turns. The controller itself is therefore quite small, because the behaviour is mainly encoded in the genome.

3 Genetic Algorithm

The genetic algorithm is implemented in `ga.py`. Each individual of the population represents one possible genome for the neural controller. At the beginning, all genomes are generated randomly, with values in the interval $[-1, 1]$.
To evaluate an individual, the Python script passes its genome to ARGoS through the `GENOME` environment variable and runs the simulation file `evaluate-controller-nn.argos`.
At the end of the simulation, the controller computes the Euclidean distance between the robot and the light source. The fitness is higher when the final distance from the light is smaller.
Each individual is evaluated more than once and the mean fitness is used. This makes the evaluation more stable. The new population is created using tournament selection, mutation and elitism. Tournament selection chooses the better individual between two randomly selected candidates. Mutation adds Gaussian noise to some genes, keeping the values inside $[-1, 1]$. Elitism keeps the best individuals of the current generation, so good solutions are not lost.

4 Conclusion

The implemented system connects a Lua neural controller with a Python genetic algorithm. The robot behaviour is not programmed with explicit rules for following the light, but it is obtained by evolving the weights of the neural network.
This activity shows that evolutionary robotics can be used to optimise a simple controller in simulation. The experiments also highlight that the choice of genetic algorithm parameters has an important impact on the final result. Mutation helps the algorithm explore new solutions, while elitism preserves the best individuals between generations. The value of n_{eval} is also important, because using more evaluations makes the fitness estimate more stable than relying on a single simulation.
The comparisons made in this lab are useful to understand the general behaviour of the algorithm, but they are still limited. A more complete analysis would require more replicas, boxplots, and statistical tests. The same approach could also be extended to collision avoidance by adding proximity sensors and modifying the fitness function to penalize collisions. For phototaxis with collision avoidance, the controller should combine both objectives: reaching the light source while moving safely in the environment.